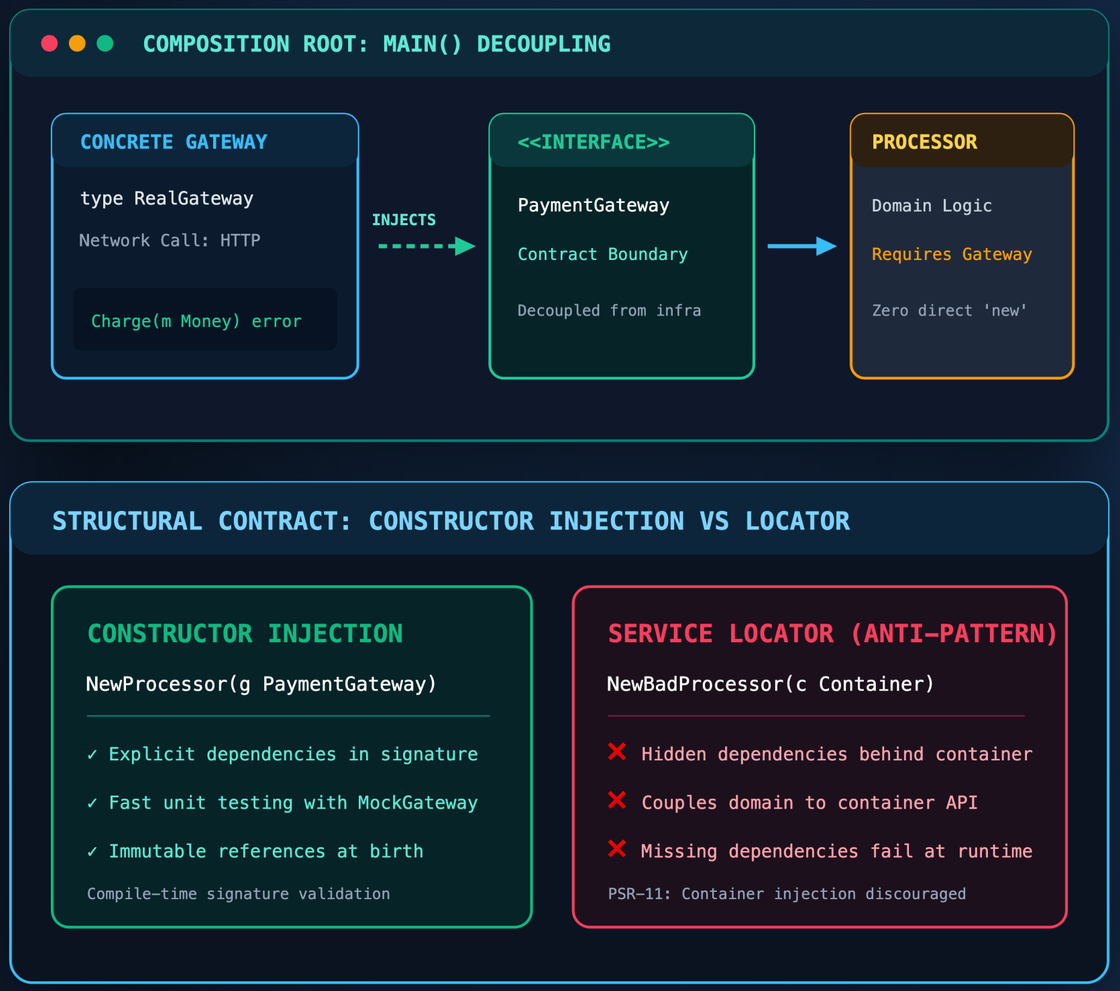
Software Engineering Lab

Lab 16

Dependency Injection & IoC

Constructor Injection vs Service Locator dalam Arsitektur Go

Lukman / lukman-ss



Hardcoded Dependency Merusak Uji Sistem

Instansiasi langsung provider eksternal di dalam domain logic membuat unit test mustahil.

MASALAH ARSITEKTUR NYATA

Processor memanggil RealGateway secara langsung di dalam method.
Unit test terpaksa membuat panggilan jaringan ke gateway eksternal.
Kegagalan koneksi pihak ketiga membuat build CI merah seketika.

Instansiasi langsung RealGateway di dalam Processor mengunci dependensi konkret ke domain. Test tidak bisa mengisolasi kegagalan gateway dari bug logika bisnis.

Hardcoded dependencies force integration tests where unit tests belong.

Service Locator Menyembunyikan Kebutuhan Komponen

Menyuntikkan container ke seluruh struct hanya memindahkan kopling ke runtime.

KELEMAHAN SERVICE LOCATOR

Signature fungsi menyembunyikan dependensi asli di balik interface Container.
Compiler tidak tahu apakah service yang dicari benar-benar ada di registry.
Missing dependency baru memicu error saat transaksi dieksekusi di runtime.

Service Locator tampak fleksibel tetapi menyembunyikan kontrak dependensi. Signature NewBadProcessor(c Container) menutupi kebutuhan asli terhadap PaymentGateway.

Service Locator trades compile-time visibility for hidden runtime lookups.

IoC Bukan Fitur, Tapi Arah Kendali

Hollywood Principle: modul bisnis menerima dependensi, bukan mencarinya sendiri.

PRINSIP INTI

Pisahkan fase konfigurasi (wiring) dari fase penggunaan (execution).
Modul domain tidak boleh mengimpor paket infrastruktur konkret.
Composition root bertindak sebagai perakit tunggal object graph.

Prinsip utama IoC adalah membalik inisiatif pemanggilan. Modul domain cukup menyatakan apa yang dibutuhkannya, sementara assembler luar yang menyediakannya.

Separate configuration from use: assemble components before running logic.

Alur Request Melalui Boundary Abstraksi

Panggilan proses pembayaran didelegasikan melalui interface PaymentGateway.

STEP 01

Client Request

Memanggil ProcessPayment(amount)

STEP 02

Processor Domain

Validasi amount > 0 & buat Money

STEP 03

PaymentGateway

Abstraksi kontrak Charge(Money)

STEP 04

Real/Mock Gateway

Eksekusi jaringan atau verifikasi test

Request masuk ke domain Processor, melewati guard validasi, lalu diteruskan ke PaymentGateway. Implementasi konkret tidak diketahui oleh domain.

Interfaces establish clear boundaries between domain rules and external side effects.

Constructor Membuat Dependency Terlihat

Signature NewProcessor menyatakan kontrak dependensi wajib secara eksplisit.

```
type Processor struct {
    gateway PaymentGateway
}
func NewProcessor(g PaymentGateway) *Processor {
    return &Processor{gateway: g}
}
```

NewProcessor memindahkan pemilihan gateway ke caller. Domain tidak lagi membuat RealGateway sendiri, sehingga test dapat menyuntikkan mock.

Explicit constructor parameters make missing dependencies impossible at compile time.

internal/di/processor.go:6-13

Jalur Kegagalan: Nil Interface Panics

Go mengizinkan passing nil ke interface; runtime panics jika constructor tidak dijaga.

STEP 01

Caller Passes Nil

NewProcessor(nil) lolos kompilasi

→

STEP 02

Constructor Unchecked

Struct menyimpan pointer gateway nil

→

STEP 03

ProcessPayment Invoked

Logika bisnis lolos validasi amount

→

STEP 04

Runtime Nil Panic

p.gateway.Charge(m) memicu panic nil pointer

> Constructor injection di Go tidak otomatis menolak nil tanpa pengecekan manual. Jika caller mengoper nil, panic baru meledak saat method dipanggil.

Interface types permit nil values: constructors still require explicit nil guards.

internal/di/processor.go:11–13

Biaya Tersembunyi: Wiring Boilerplate

Manual injection menjaga kejernihan arsitektur namun menambah kode perakitan.

Aspek	Manual Wiring (Lab Ini)	DI Framework (Wire/Dig)
Dependency Graph	Terbaca di main.go	Tersembunyi di reflection / gen
Compile Overhead	Zero overhead	Code-gen build step tambahan
Refactoring Safety	Compiler menangkap perubahan	Bergantung pada framework container

Manual wiring menambah boilerplate pada composition root. Ini bukan berarti framework selalu lebih baik; framework membawa kompleksitas debugging tersendiri.

Boilerplate in the composition root is the explicit cost of architectural transparency.

Setup Uji: Isolasi Dua Jalur Arsitektur

Menguji perilaku validasi dan propagasi error pada Processor dan BadProcessor.

6 Test Cases

Unit Suite

3 Processor • 3 BadProcessor

MockGateway

Test Double

In-memory state inspection

Go 1.22

Race Detector

go test -race ./...

Eksperimen membandingkan 6 skenario uji: transaksi valid, gateway error, dan input negatif pada kedua varian processor.

Symmetric test coverage isolates architectural trade-offs under identical conditions.

Output Demo: Verifikasi Eksekusi Terpisah

Kedua pola memproses pembayaran dengan implementasi RealGateway yang sama.

```
$ go run ./cmd/demo
--- Running Constructor Injection ---
RealGateway charging 100 USD
--- Running Service Locator ---
RealGateway charging 200 USD
```

Demo membuktikan kedua pola menghasilkan output fungsional yang sama. Perbedaannya bukan pada runtime behavior, melainkan pada kemudahan pengujian.

Identical runtime output does not imply identical architectural maintainability.

Test Memvalidasi Alur, Bukan Kesiapan Skala

Keberhasilan mock membuktikan isolasi logika, bukan keandalan jaringan riil.

YANG DIBUKTIKAN TEST

Processor mendelegasikan pembayaran ke MockGateway tanpa jaringan.
Validasi amount <= 0 menolak eksekusi sebelum memanggil gateway.
Race detector bersih pada seluruh skenario pengujian unit.

Ketahanan koneksi HTTP saat gateway riil mengalami timeout.

Manajemen connection pool dan lifecycle pada beban production.

Keamanan race condition pada shared state antar-goroutine.

Passing mock tests prove logic isolation; production still requires resilience tests.

Trade-off: Fleksibilitas vs Kejelasan Kontrak

Service locator memudahkan penambahan service baru tetapi menutupi kontrak.

CONSTRUCTOR INJECTION

Kontrak terlihat jelas pada signature konstruktor.
Compiler menolak caller jika argumen kurang.
Constructor berisiko panjang jika kelas terlalu sibuk.

Signature konstruktor tetap ringkas menerima Container tunggal.

Kebutuhan dependensi tersembunyi di dalam implementasi method.

Kompilasi lolos meski service belum terdaftar di container.

Explicit constructor parameters protect architectural clarity at the cost of signature length.

Value Object Tidak Memerlukan Injeksi

Struktur data tanpa dependensi infrastruktur cukup diinstansiasi langsung.

```
func (p *Processor) ProcessPayment(amount int) error {
    if amount <= 0 {
        return errors.New("invalid amount")
    }
    m := Money{Amount: amount, Currency: "USD"}
    return p.gateway.Charge(m)
}
```

Money adalah value object murni tanpa efek samping. Menyuntikkan factory untuk struct sederhana seperti Money adalah over-engineering yang tidak perlu.

Reserve dependency injection for behavioral services, not pure value objects.

Sebelum vs Sesudah Constructor Injection

Mengubah instansiasi internal menjadi penyuntikan kontrak eksternal.

BEFORE: NAIVE / UNPROTECTED

HARDCODED REAL GATEWAY

- Processor membuat new RealGateway() di method.
- Test terpaksa memanggil network eksternal.
- Gagal ganti provider tanpa mengubah domain code.

AFTER: OPTIMIZED / RESILIENT

CONSTRUCTOR INJECTION

✓ Processor menerima interface PaymentGateway.

✓ Test menyuntikkan MockGateway in-memory seketika.

✓ Caller bebas menukar provider di composition root.

Externalizing construction decouples business logic from concrete infrastructure.

Batasan Lab: Manual Wiring In-Memory

Lab ini membuktikan struktur injeksi murni tanpa framework otomatis.

CAKUPAN IMPLEMENTASI

Manual composition root di cmd/demo/main.go.
Single interface PaymentGateway dan implementasi mock.
Validasi domain sebelum delegasi ke infrastruktur.

DI framework otomatis seperti Wire, Dig, atau Fx.

Lifecycle management: Singleton, Scoped, dan Transient.

Scope validation untuk mencegah memory leak antar-request.

Manual wiring demonstrates the core principle without the complexity of automated frameworks.

Kasus Batas: Penanganan Parameter Tidak Valid

Validasi domain wajib memblokir eksekusi sebelum menyentuh gateway.

```
proc := di.NewProcessor(mock)
err := proc.ProcessPayment(-10)
if err == nil {
    t.Fatalf("expected error for negative amount")
}
if mock.ChargedMoney.Amount != 0 {
    t.Errorf("gateway should not have been called")
}
```

Test memastikan ProcessPayment(-10) gagal seketika dan ChargedMoney.Amount tetap 0. DI tidak meniadakan validasi input di trust boundary.

Validate input at domain boundaries before delegating to injected infrastructure.

tests/processor_test.go:47–59

Fakta yang Dibuktikan Suite Pengujian

Enam test case memverifikasi perilaku terisolasi pada kedua pola arsitektur.

FAKTA TERVERIFIKASI

Processor dan BadProcessor sukses mendelegasikan transaksi valid.
Error dari PaymentGateway dipropagasikan utuh ke caller.
Input negatif ditolak tanpa memanggil gateway eksternal.
go test -race tidak mendeteksi data race pada skenario uji.

Suite pengujian membuktikan bahwa dependency mocking berhasil memisahkan logika domain dari efek samping jaringan pada semua kasus yang diuji.

Clean unit tests confirm structural wiring under specified test fixtures.

Batas Integritas: Apa yang Tidak Diuji

Hasil unit test bersih bukan jaminan arsitektur concurrency production aman.

DI LUAR BUKTI LAB

Bukan jaminan zero-overhead pada graf dependensi kompleks.
Belum membuktikan keandalan retries saat gateway riil timeout.
Race detector hanya membaca jalur pengujian yang dijalankan.
Tidak membuktikan efisiensi memory lifecycle skala production.

Test suite yang lolos hanya membuktikan skenario mock yang dirancang. Jangan menyamakan clean unit test dengan kesiapan sistem menghadapi traffic production.

Unit test assertions verify local contracts, not distributed system stability.

Risiko: Constructor Over-Injection

Constructor dengan terlalu banyak dependensi menandakan pelanggaran SRP.

GEJALA KERUSAKAN DESAIN

Constructor menerima belasan parameter interface berbeda.
Kelas mengurusi terlalu banyak domain logic (God Object).
Setiap unit test memerlukan belasan mock setup yang rapuh.
Solusi: pecah service menjadi komponen yang lebih granular.

Menumpuk banyak dependensi ke dalam satu constructor bukan salah DI, melainkan sinyal bahwa class tersebut memikul terlalu banyak tanggung jawab.

A constructor with too many dependencies is a code smell for single-responsibility violation.

Panduan Implementasi DI yang Aman

Langkah praktis menerapkan dependency injection pada codebase Go.

- ✓ Definisikan interface di sisi consumer, bukan di sisi provider.
- ✓ Gunakan Constructor Injection sebagai default untuk seluruh service.
- ✓ Tambahkan guard validasi nil pada setiap konstruktor.
- ✓ Pusatkan instansiasi konkret di composition root (main.go).

Define interfaces at consumption boundaries and centralize wiring in the composition root.

Anti-Pattern: Passing Container ke Domain

Menyuntikkan service locator merusak enkapsulasi dan transparansi kontrak.

ALASAN MENGHINDARI LOCATOR

PSR-11 dan standar industri menegaskan: 'Users SHOULD NOT pass container'.
Domain terikat pada interface container milik framework.
Mocking menjadi sulit karena seluruh container harus ditiru.
Kompilasi berhasil meski service target belum terdaftar.

Menyuntikkan container ke domain logic membuat komponen terikat pada framework dan mengaburkan kebutuhan sistem sesungguhnya.

Never inject the container: pass only the specific service required by the domain.

Reproduksi Mandiri di Mesin Lokal

Eksperimen dijalankan dengan Go standard library tanpa dependency eksternal.

```
cd labs/16-dependency-injection
go test -v ./...    # validasi 6 test cases
go test -race ./... # verifikasi race detector
go run ./cmd/demo   # simulasi DI vs Service Locator
```

Lab ini sepenuhnya zero-dependency dan dapat direproduksi langsung. Eksekusi test dan demo memverifikasi pemisahan konstruksi dan penggunaan secara deterministik.

Standard library tests ensure deterministic verification without external tooling overhead.

DI Memperlihatkan Dependency Komponen

Constructor injection memindahkan keputusan perakitan ke pemanggil.

PRINSIP REKAYASA

Constructor injection membuat kontrak ketergantungan eksplisit.
Validasi nil tetap menjadi tanggung jawab konstruktor.
Pemisahan konstruksi dari penggunaan memungkinkan pengujian terisolasi.

Nilai sejati DI bukan pada framework otomatis, melainkan pada kejelasan kontrak arsitektur. Memisahkan konstruksi dari penggunaan membuat sistem dapat diuji dan berevolusi.

Dependency injection makes dependencies visible; it does not replace runtime validation.

Constructor membuat dependency terlihat; domain tetap independen.

Constructor injection exposes dependencies explicitly; configuration remains external to use.

Software Engineering Lab 16 – Dependency Injection & IoC

internal/di/gateway.go • internal/di/processor.go
internal/di/locator.go • cmd/demo/main.go
tests/processor_test.go • README.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/16-dependency-injection`

Lukman / lukman-ss

Sumber: commit 76f0059
Test passing: 6 test cases, 0 failures.
Race detector: clean (go test -race ./...)
Manual in-memory wiring; bukan proof framework DI production completeness.